

Codeforces 1543A - Exciting Bets

Problem Statement

You are given two integers a and b .

In one move, you can increase one number by 1 and decrease the other by 1 (equivalently, the solution boils down to finding the optimal common shift relative to their fixed difference).

You need to find:

- The maximum possible GCD of the two numbers.
- The minimum number of moves required to achieve that GCD.

Observation 1: What determines the GCD?

Suppose after some moves, the numbers become x and y . Let their GCD be g . Since g divides both numbers:

$$g \mid x$$

$$g \mid y$$

then g must also divide their difference. Because $y - x$ is always divisible by g . This comes from the property: *If a number divides two integers, it also divides their difference.*

Observation 2: What is the largest possible GCD?

Let $a = 8$ and $b = 20$. Their difference is:

$$20 - 8 = 12$$

Since every possible GCD must divide the difference, the **Maximum GCD** ≤ 12 .

Can we actually make the GCD equal to 12? Yes. For example, **12** and **24** have $\text{gcd} = 12$. So:

$$\text{Maximum GCD} = |a - b|$$

This is the most important observation in the problem.

Observation 3: What if the numbers are equal?

Suppose $a = 10$ and $b = 10$. Their difference is **0**. There is no larger GCD to obtain because the numbers are already equal. So the answer is **0 0**. That's why your code first checks:

```
if(reqGcd == 0){
    cout<<0<<" "<<0<<endl;
    continue;
}
```

Now the only remaining task

We already know **Maximum GCD = reqGcd**. Now we only need to find: *How many moves are required to make both numbers divisible by reqGcd?*

Why do we only check one number?

Suppose $a = 8$, $b = 20$, and $reqGcd = 12$. Notice that $20 = 8 + 12$. If we make 8 a multiple of 12 , then 20 automatically becomes a multiple of 12 because it differs by exactly 12 . For example:

$$8 \rightarrow 12$$

$$20 \rightarrow 24$$

Both are divisible by 12 . So we only need to calculate the moves for one number. That's why your code only uses `a % reqGcd` and never checks `b`.

How do we know how many moves are needed?

Let's look at $a = 29$ and $reqGcd = 12$. The multiples of 12 are $0, 12, 24, 36, 48$.

24 ----- 29 ----- 36

Visualizing the positioning of 'a' on the number line relative to multiples of reqGcd

There are only two nearest multiples:

- **Previous multiple:** $24 \rightarrow$ Distance: $29 - 24 = 5$
- **Next multiple:** $36 \rightarrow$ Distance: $36 - 29 = 7$

Obviously we choose 5 moves.

Why does `a % reqGcd` give the previous distance?

Your code states:

```
long long moves_prevMultiple = a % reqGcd;
```

Suppose $29 \% 12 = 5$. What does this remainder actually mean? It means $29 = 24 + 5$, where 24 is the previous multiple. So the remainder is exactly the distance from the previous multiple. Therefore, `moves_prevMultiple = a % reqGcd;` is literally the *distance to the previous multiple*.

Why is the next distance `reqGcd - remainder`?

Current number: 29

Next multiple: 36

Distance: $36 - 29 = 7$

Formula: $12 - 5 = 7$

Why? Because $29 = (2 \times 12) + 5$. The next multiple is 3×12 . The distance is:

$$3 \times 12 - (2 \times 12 + 5) = 12 - 5 = 7$$

So `long long moves_nextMultiple = reqGcd - moves_prevMultiple;` always gives the *distance to the next multiple*.

Why do we take the minimum?

There are only two possible directions. Move left ($29 \rightarrow 24$) takes 5 moves. Move right ($29 \rightarrow 36$) takes 7 moves. Naturally, the minimum is 5. That's why:

```
long long minMoves = min(moves_prevMultiple, moves_nextMultiple);
```

Understanding your code line by line

```
long long reqGcd = abs(a-b);
```

Finds the maximum possible GCD.

```
if(reqGcd==0){  
    cout<<0<<" "<<0<<endl;  
    continue;  
}
```

Handles the case when both numbers are already equal.

```
long long moves_prevMultiple = a % reqGcd;
```

Computes the distance to the previous multiple of reqGcd.

```
long long moves_nextMultiple = reqGcd - moves_prevMultiple;
```

Computes the distance to the next multiple of reqGcd.

```
long long minMoves = min(moves_prevMultiple, moves_nextMultiple);
```

Chooses the smaller direction.

```
cout<<reqGcd<<" "<<minMoves;
```

Prints Maximum GCD and Minimum moves.

Complete Dry Run

Suppose $a = 10$ and $b = 22$.

Difference: $reqGcd = 12$. Maximum possible GCD is 12 .

Now make 10 divisible by 12. Multiples are $0, 12, 24$.

0 - - - - - 10 - - 12 - - - - - 24

Dry Run visualization on the number line

Previous distance is 10 because $10 \% 12 = 10$.

Next distance is $12 - 10 = 2$.

We choose 2 .

After adding 2, the shifts occur as follows: $10 \rightarrow 12$ and $22 \rightarrow 24$. Now, $gcd(12, 24) = 12$. The output will be `12 2`.

Key Takeaways (Remember These)

- The maximum possible GCD is always the absolute difference $|a - b|$. Any common divisor must divide the difference, and this bound is achievable.
- If $a == b$, the answer is $0\ 0$. There's no larger GCD to achieve.
- You only need to make one number divisible by $|a - b|$. The other number will automatically become divisible because their difference is exactly $|a - b|$.
- $a \% g$ is the distance to the previous multiple of g .
- $g - (a \% g)$ is the distance to the next multiple of g .
- The minimum of these two distances is the minimum number of moves.

Interview Intuition (30 seconds)

The key observation is that any common divisor of two numbers must divide their difference. Therefore, the largest possible GCD is the fixed difference $|a - b|$. Once we know this target GCD, we only need to make one number a multiple of it. The remainder $a \% g$ tells us how far the number is from the previous multiple, while $g - (a \% g)$ gives the distance to the next multiple. The smaller of these two distances is the minimum number of moves required.